

Tecnológico de Monterrey

Evidencia 2 – Actividad Integral de Estructuras de Datos Lineales

Ian Armando Borde Escobar - A00846007

18 de octubre de 2026

Evidencia de ejecución

Lista ligada simple

```
[]
OK: Lista inicialmente vacia
OK: Eliminar en vacio no interrumpe
[10]
OK: Insercion de un elemento
OK: Reinsercion despues de eliminar el unico elemento
[10, 20, 30, 40]
OK: Tail actualizado al borrar por indice
[20, 30, 60]
OK: Eliminacion al inicio
```

Pila

```
Mensaje: La pila esta vacia
OK: Pop en vacio muestra mensaje y permite continuar
OK: Pila con un elemento
OK: Pop del unico elemento
Orden LIFO esperado: 30 20 10
Pop: 30
Pop: 20
Pop: 10
OK: Pila vacia al terminar
```

Fila

```
Mensaje: La fila esta vacia
OK: Dequeue en vacio muestra mensaje y permite continuar
OK: Fila con un elemento
OK: Dequeue del unico elemento
Orden FIFO esperado: 10 20 30
Dequeue: 10
Dequeue: 20
Dequeue: 30
OK: Fila vacia al terminar
```

Pruebas de estructuras y carga

Lista doblemente ligada

OK: Lista doble inicialmente vacia
OK: Eliminar nodo nulo no interrumpe
OK: Lista doble con un elemento
[20]
OK: Eliminar unico nodo actualiza ambos extremos
Adelante esperado: [10, 20, 30, 40, 50]
[10, 20, 30, 40, 50]
Atras esperado: [50, 40, 30, 20, 10]
[50, 40, 30, 20, 10]
Tras borrar inicio, final e intermedio: [20, 40]
[20, 40]
[40, 20]
OK: Eliminacion en extremos e intermedio

Carga y validación

Ambos archivos cargaron 6818 registros y omitieron cero líneas

El nombre vacío y un archivo inexistente mostraron un aviso y permitieron volver a intentar la carga

Un archivo temporal con nueve líneas cargó dos registros válidos y omitió siete, reportando el número y motivo de cada línea incorrecta

Linea 2: 29 de febrero de 2023 inexistente
Linea 3: mes Xxx invalido
Linea 4: hora 24:00:00 fuera de rango
Linea 5: IP 256.0.0.1 invalida
Linea 6: descripcion ausente
Linea 7: linea vacia
Linea 8: registro incompleto

Carga y líneas inválidas

Captura de la salida obtenida en la nueva corrida con nombre vacío, archivo inexistente y siete líneas inválidas



```
PROCESAMIENTO DE LOGS DE RED

Archivo que deseas cargar:
log607-1.txt
log607-2.txt
Archivo: El nombre del archivo no puede estar vacio

Archivo que deseas cargar:
log607-1.txt
log607-2.txt
Archivo: No se pudo abrir el archivo: no_existe_para_prueba.log

Archivo que deseas cargar:
log607-1.txt
log607-2.txt
Archivo: Linea 2 omitida: Dia invalido para el mes y anio indicados
Linea 3 omitida: Mes invalido; utiliza Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov o Dec
Linea 4 omitida: Hora fuera de rango; utiliza 00:00:00 a 23:59:59
Linea 5 omitida: IP invalida; se esperan cuatro numeros entre 0 y 255
Linea 6 omitida: Falta la descripcion del evento
Linea 7 omitida: Linea vacia
Linea 8 omitida: Faltan campos de fecha, hora o IP

RESUMEN DE CARGA
Archivo: invalidos.log
Registros cargados: 2
Lineas omitidas: 7
```

Opción 1 Ordenar por fecha y hora

Ordenamiento de log607-1.txt

```
RESULTADOS DEL ORDENAMIENTO
Algoritmo: Merge Sort estable
Archivo: data/log607-1.txt
Registros: 6818
Prediccion: n * log2(n)
Tiempo de ejecucion: 5.152625 ms
Comparaciones del ordenamiento: 78289
Profundidad maxima de recursion: 14
Coincide con la prediccion: Si
Archivo generado: out/output608.out
```

Ordenamiento de log607-2.txt

```
RESULTADOS DEL ORDENAMIENTO
Algoritmo: Merge Sort estable
Archivo: data/log607-2.txt
Registros: 6818
Prediccion: n * log2(n)
Tiempo de ejecucion: 2.908291 ms
Comparaciones del ordenamiento: 49551
Profundidad maxima de recursion: 14
Coincide con la prediccion: Si
Archivo generado: out/output608.out
```

Archivo de salida

```
out/output608.out
Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication
Sep 08 2024 04:37:37 10.14.248.79 Configuration errors
Sep 08 2024 04:44:15 10.14.135.12 Application error
Sep 08 2024 05:50:57 10.14.22.235 Changes in File Integrity
Sep 08 2024 10:42:42 10.14.40.195 Data corruption
Sep 08 2024 10:47:52 10.14.24.223 Data corruption
Sep 08 2024 15:23:16 10.14.206.243 Weak or easily guessable passwords
Sep 08 2024 15:25:34 10.14.220.55 Broken authentication and session management
```

Opciones 2 y 3 Consultas por día

Opción 2 Sep 14 2024 con 12 registros encontrados

```
Logs del dia Sep 14 2024
Sep 14 2024 00:24:23 10.14.115.26 Social engineering attacks
Sep 14 2024 00:25:20 10.14.177.24 Suspicious IP Addresses
Sep 14 2024 00:52:28 10.14.175.238 Security breaches
Sep 14 2024 01:18:02 10.14.66.138 Anomalous User Activity
Sep 14 2024 05:14:25 10.14.85.112 Data Exfiltration Attempts
Sep 14 2024 05:15:49 10.14.58.71 Port Scanning Activity
Sep 14 2024 11:57:05 10.14.197.208 Cross-Site Scripting
Sep 14 2024 18:34:13 10.14.65.141 Anomalous User Activity
Sep 14 2024 20:00:03 10.14.47.235 Data Exfiltration Attempts
Sep 14 2024 22:23:31 10.14.141.44 Failed Login Attempts
Sep 14 2024 23:03:07 10.14.175.98 Port Scanning Activity
Sep 14 2024 23:30:59 10.14.41.52 Suspicious IP Addresses
Total encontrado: 12
```

Opción 3 Los mismos registros en orden descendente

```
Logs del dia Sep 14 2024 en orden descendente
Sep 14 2024 23:30:59 10.14.41.52 Suspicious IP Addresses
Sep 14 2024 23:03:07 10.14.175.98 Port Scanning Activity
Sep 14 2024 22:23:31 10.14.141.44 Failed Login Attempts
Sep 14 2024 20:00:03 10.14.47.235 Data Exfiltration Attempts
Sep 14 2024 18:34:13 10.14.65.141 Anomalous User Activity
Sep 14 2024 11:57:05 10.14.197.208 Cross-Site Scripting
Sep 14 2024 05:15:49 10.14.58.71 Port Scanning Activity
Sep 14 2024 05:14:25 10.14.85.112 Data Exfiltration Attempts
Sep 14 2024 01:18:02 10.14.66.138 Anomalous User Activity
Sep 14 2024 00:52:28 10.14.175.238 Security breaches
Sep 14 2024 00:25:20 10.14.177.24 Suspicious IP Addresses
Sep 14 2024 00:24:23 10.14.115.26 Social engineering attacks
Total encontrado: 12
```

Opción 4 Estadísticas

```
Total de registros: 6818

Dias con mas incidentes
Jan 27 2025 | Apariciones: 21

Tipos de evento mas frecuentes
Malware Detection Alerts | Apariciones: 342

Direcciones IP mas frecuentes
10.14.123.164 | Apariciones: 3
10.14.126.198 | Apariciones: 3
10.14.161.42 | Apariciones: 3
10.14.162.205 | Apariciones: 3
10.14.176.221 | Apariciones: 3
10.14.186.139 | Apariciones: 3
10.14.199.224 | Apariciones: 3
10.14.200.76 | Apariciones: 3
10.14.242.186 | Apariciones: 3
10.14.29.229 | Apariciones: 3
10.14.36.7 | Apariciones: 3
10.14.69.71 | Apariciones: 3
10.14.94.165 | Apariciones: 3
```

Empates en las tres categorías

```
Total de registros: 4

Dias con mas incidentes
Sep 14 2024 | Apariciones: 2
Sep 15 2024 | Apariciones: 2

Tipos de evento mas frecuentes
Evento A | Apariciones: 2
Evento B | Apariciones: 2

Direcciones IP mas frecuentes
10.0.0.1 | Apariciones: 2
10.0.0.2 | Apariciones: 2
```

Casos de borde y salida

```
MENU PRINCIPAL
1. Ordenar por fecha y hora
2. Buscar logs por dia
3. Mostrar logs por dia en orden descendente
4. Estadisticas del archivo
5. Salir
Opcion: Primero ordena los registros con la opcion 1
```

```
Dia a buscar, por ejemplo Sep 14 2024:
Logs del dia Sep 14 2024 en orden descendente
Sep 14 2024 12:00:00 10.0.0.2 Evento B
Sep 14 2024 10:00:00 10.0.0.1 Evento A
Total encontrado: 2
```

```
Dia a buscar, por ejemplo Sep 14 2024:
Formato invalido Usa Mmm DD AAAA
Dia a buscar, por ejemplo Sep 14 2024:
Mes invalido Usa Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov o Dec
Dia a buscar, por ejemplo Sep 14 2024:
Dia invalido para el mes y anio indicados
Dia a buscar, por ejemplo Sep 14 2024:
Dia invalido para el mes y anio indicados
Dia a buscar, por ejemplo Sep 14 2024:
El dia debe tener uno o dos digitos y el anio cuatro digitos
Dia a buscar, por ejemplo Sep 14 2024:
Formato invalido Usa Mmm DD AAAA
Dia a buscar, por ejemplo Sep 14 2024:
```

```
Logs del dia Jan 01 1900
No se encontraron registros para ese dia
Total encontrado: 0
```

```
La lista ya esta ordenada por fecha y hora
Archivo generado: out/output608.out

Opcion: Programa finalizado
```

Resultados del ordenamiento

Medición	**log607-1.txt**	**log607-2.txt**
Registros	6818	6818
Predicción	$n \cdot \log_2(n)$	$n \cdot \log_2(n)$
Tiempo en ms	5.152625	2.908291
Comparaciones	78289	49551
Profundidad máxima	14	14
Conservó el orden en empates	Sí	Sí
Coincidió con la predicción	Sí	Sí

Interpretación

En estas corridas, log607-2.txt necesitó menos comparaciones y tuvo un tiempo menor, el orden inicial influye en las comparaciones de la mezcla, aunque los dos archivos contienen 6818 registros

La profundidad máxima fue 14 en ambos casos porque Merge Sort divide la lista aproximadamente a la mitad, esta profundidad indica los niveles simultáneos de llamadas recursivas y crece en O(log n)

Las salidas completas coincidieron con un ordenamiento estable de referencia externo a la aplicación, se conservaron todos los registros y el orden de llegada en los 18 grupos con fecha y hora repetidas de cada archivo

La predicción corresponde al comportamiento teórico O(n log n), dos archivos del mismo tamaño no demuestran por sí solos el crecimiento asintótico y los tiempos pueden variar entre ejecuciones

Prompts

¿Cómo puedo ordenar mi lista doblemente ligada con Merge Sort sin copiar los registros a un arreglo y conservar el orden de llegada cuando tienen la misma fecha y hora?

Me ayudo a ordenar los nodos mediante enlaces y conservar el orden de llegada cuando hay empates y adapte esto a mi lista doblemente ligada

¿Cómo puedo mostrar todos los empates en las estadísticas y comprobar con un archivo pequeño que no muestro solamente el primer máximo?

Me ayudo a contar las apariciones y mostrar todos los elementos empatados, lo comprobé con un archivo pequeño con empates en días eventos e ip

Ando intentando compilar desde VS Code pero salen muchos errores en main.cpp ¿Por qué no compila y como puedo solucionarlo?

Me ayudo a identificar que debia compilar main.cpp con LogRecord.cpp, yo lo intentaba ejecutar con el compilador de vs code arriba y me ayudo con el comando de compilación para incluir ambos archivos

Sugerencia innecesaria y corrección

Chat cuando le pedi que checara que todo el entregable estaba de acuerdo a la rubica y andaba checando que todo estuviera bien agregó hasStableOrder y varias comprobaciones adicionales después, agrego extras sin que se lo pidiera por usar un prompt muy general y ambiguo y lo detecté al comparar las instrucciones y pedí que eliminara todo lo que arreglo

Reflexión personal

En esta actividad relacioné las estructuras de datos con tareas concretas de una aplicación, partí de estructuras que ya tenía, las adapté, preparé pruebas e integré el programa, la IA me ayudó con las validaciones, las búsquedas y las estadísticas, también vi que puede agregar funciones innecesarias, como las comprobaciones adicionales de orden y estabilidad, al comparar la propuesta con las instrucciones pedí retirar esos extras y conservé las verificaciones durante las pruebas y sin la ayuda de la IA habría dedicado más tiempo a consultar ejemplos y probar cada función y hacer validaciones

La lista ligada simple es la base de la Pila y la Fila, en la Pila se inserta y elimina al inicio para obtener un comportamiento LIFO, en la Fila se inserta al final y se elimina al inicio para obtener FIFO, head y tail permiten realizar estas operaciones en O(1), la Fila procesa las líneas en orden de llegada, después de desencolar cada una se valida y convierte en un registro, las líneas incorrectas se reportan y las válidas se guardan al final de la lista doblemente ligada

Elegí Merge Sort porque permite ordenar la lista mediante sus enlaces, sin copiar los registros a un arreglo, su comportamiento es O(n log n) y conserva el orden de llegada en los empates al elegir primero el nodo izquierdo durante la mezcla

Para la opción 3 utilicé una Pila de apuntadores a las coincidencias, push y pop permiten verlas en orden inverso, aunque también podía localizar el último registro del día y regresar usando prev, la Pila cumple el requisito y hace explícito el comportamiento LIFO, pero necesita memoria adicional O(k), recorrer con prev evita esa memoria, aunque requiere localizar el último registro y detenerse cuando cambie la fecha

La búsqueda binaria necesita consultar el elemento intermedio, en una lista ligada llegar a él requiere recorrer enlaces, por eso no se obtiene directamente el tiempo O(log n) de un arreglo, para hacerlo posible podría construir un índice de apuntadores, con un costo de O(n) en tiempo y memoria, después buscaría los límites del día en O(log n), pero tendría que mantener ese índice si cambia la lista

Ambos archivos tenían 6818 registros, el primero necesitó 78289 comparaciones y el segundo 49551, el segundo también terminó más rápido, esto muestra que estar casi ordenado no significa menos comparaciones en esta implementación, ambos alcanzaron una profundidad de 14 niveles de llamadas recursivas, cada llamada ocupa espacio en la pila de llamadas, distinta de mi Pila de la opción 3, la profundidad crece en O(log n) porque Merge Sort divide la lista aproximadamente a la mitad